HPCA 2026

DSASSASSIN

Cross-VM side-channel attacks by exploiting Intel’s Data Streaming Accelerator

Ben Chen · Kunlin Li · Shuwen Deng · Dongsheng Wang · Yun Chen

HKUST (Guangzhou) · Tsinghua University

Presented by [your name] · ECE-MAR · 27 October 2026

The wall has a crack

Address isolation still leaves the shared activity visible



The leakage is about resource use — not access to victim memory.

The wall has a crack

Four demonstrations, one humble signal

17.19 kbps

Covert channel

DevTLB · 4.63% bit error

85.73%

Website fingerprinting

100 sites · 1% baseline

98.4%

Keystroke timing F1

SWQ · 1.21 ms deviation

98.6%

LLM fingerprinting

Validation · 8 models

None of these reads a byte of victim memory.

Paper § VI, Tables III–IV · pp. 9–12 · reported results

The route — from shared state to application leakage

	THEME	COVERS	TIME
I	The wall has a crack	thesis + payoff	4 min
II	Meet the machine	architecture + threat model	8 min
III	The detective story	reverse engineering + two primitives	16 min
IV	What it steals	covert channel + three attacks	12 min
V	Does it hold up?	robustness, comparison, defenses	6 min
VI	My verdict	strengths, limits, open questions	5 min

A running theme: VT-d guards addresses, not the accelerator’s shared heartbeat.

Act 2 of 6

02

Meet the machine

What DSA is and how a job flows through it

The device-side translation cache (DevTLB)

The honest threat model — and two kinds of sharing

Memory offload creates a new place to observe

Common operations

memcpy · memset
compare · checksums

→

DSA offload

descriptor submission
asynchronous execution

→

Observable state

translation cache
queue occupancy

Moving work off the CPU moves the side-channel surface with it.

A DSA operation is a descriptor-driven pipeline



The attacker owns two handles —

“was my submission accepted?” · “how long did my completion take?”

Observe request acceptance, or observe how long completion takes.

The device keeps its own translation cache

Device request

virtual address + PASID

→

DevTLB / ATC

hit → reuse translation

→

Host memory

physical transaction

Miss → Translation Agent → page-table walk

A cached translation stays locked — but its replacement leaks activity.

The attacker needs DSA access and co-location

Assumed	Not needed
DSA-enabled Xeon + IDXD + VT-d scalable mode	A shared CPU core
Victim actually uses DSA	Shared victim memory
Attacker may submit descriptors (CAP_SYS_RAWIO)	Permission to read victim addresses
Shares the engine (DevTLB) or the SWQ	Root / authority to reconfigure host DSA
May set its own engine / queue affinity	

“Unprivileged” still depends on pre-provisioned device access and co-location.

Two channels cross two different boundaries

DevTLB timing channel

Victim and attacker may use
different work queues —
if they share one engine.

Different queues can still interfere.

SWQ contention channel

Victim and attacker must
share the same work queue.

The same queue must be shared.

Separate work queues are not enough if the engine is shared.

Reverse engineering answers three questions

01

What is shared?

Capacity, indexing, replacement policy, PASID behaviour

02

What is visible?

Privileged counters vs. user-observed latency and flags

03

What can it reveal?

Eviction → activity

Admission → occupancy

Each microbenchmark links an implementation behaviour to an observable signal.

Characterization counters ≠ attack signals

Reverse engineering · privileged PMU	Attack signal · unprivileged
EV_ATC_ALLOC	DevTLB · own completion time (rdtsc)
EV_ATC_NO_ALLOC	SWQ · ENQCMD result via EFLAGS.ZF
EV_ATC_HIT_PREV	

Platforms: Xeon Platinum 8468V (local) · 8475B (Alibaba cloud) · Linux 6.11 · IDXD · VT-d scalable mode · SR-IOV (DevTLB setup) / Scalable IOV (shared SWQ)

PMU access supports the explanation; it is not needed for the attack loop.

Act 3 of 6

03

The detective story

Each microbenchmark rules out an alternative

Five one-entry slots — indexed by engine and field, not PASID

Prime → Idle → Probe and Congest → Idle → Probe

Clue #1 — the opcode chooses which slot you touch

OPERATION	ADDRESS FIELDS	COMPLETION	WHY USEFUL
noop	none	comp	Minimal timing probe — the scalpel
memcmp / compare	src + src2	comp	Two independent reads
memcpy	src + dst	comp	Read / write separation
dualcast	src + dst + dst2	comp	Multiple destination slots

Byte 24 is src2 for compare but dst for copy — same bits, different meaning.

The opcode changes field meaning; field meaning changes which slot is used.

The reveal — five drawers, one translation each

Engine 0

src	src2	dst	dst2	comp
one page translation	one page translation	one page translation	one page translation	one page translation

A single different page in a field’s role replaces the one translation in that slot — no eviction set required.

One-slot replacement makes a neighbour’s activity trivial to detect.

Proof of “one slot” — base → offset → base

1

Probe base

Load one page translation

→

2

Probe base + offset

Keep or replace it

→

3

Probe base again

Did it survive?

OFFSET	FINAL BASE OBSERVATION	INTERPRETATION
< 4 KiB	Hit	Same page retained
≥ 4 KiB	Miss	A different page replaced it

A single different page is enough to evict the tested field’s translation.

The drawers are independent — even sharing bits

Independent fields

copy(src A, dst D)
copy(src B, dst D)

Source replaced; destination retained
→ separate slots.

Shared encoding, distinct slots

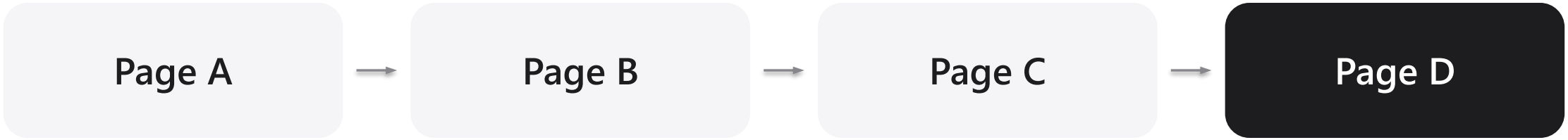
compare(src A, src2 X)
copy(src A, dst X)

Only the src reuse
produces the shared hit.

The organisation follows field semantics, including src2 versus dst.

A slot remembers the last page, not the history

One descriptor crossing several source pages



Retained in the src slot: Page D

The batch fetcher's own descriptor reads and completion-record writes did not populate these slots — a separate translation path.

A slot records recent field-specific state, not a page-access history.

The detective story

The money shot — eviction follows the engine, not the queue

E0

same WQ · same engine

Miss

Cross-VM interference

E1

different WQ · same engine

Miss

Queue separation insufficient

E2

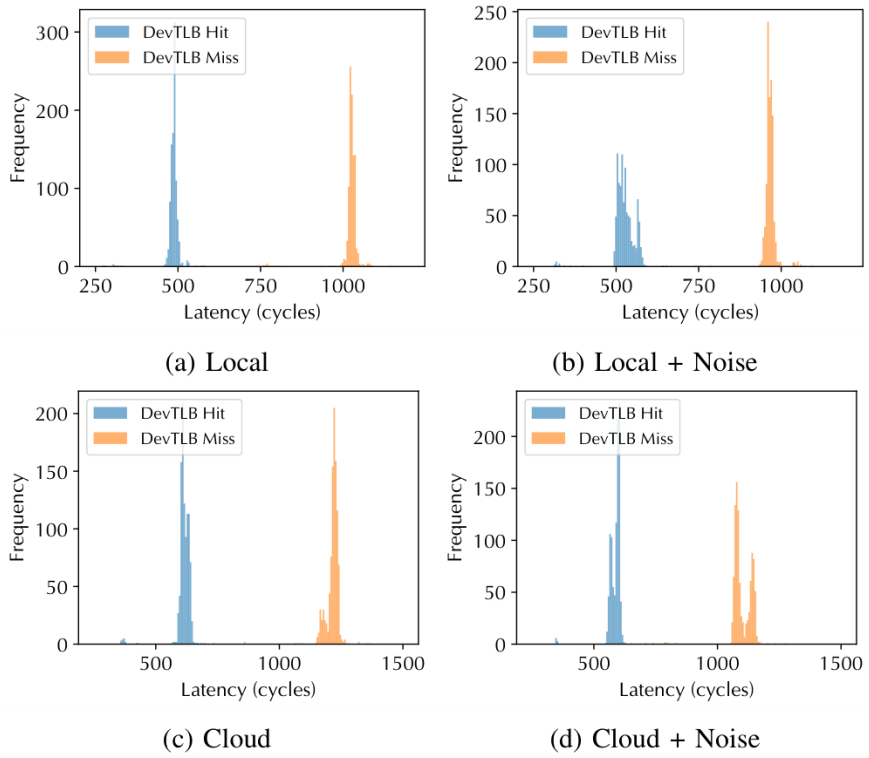
different WQ · different engine

Hit

No cross-engine eviction

E2 is the control that proves it: engine sharing is necessary and sufficient.

The clock still works — even under noise



Measured signal

Hit	≈ 500 cycles
Miss	≈ > 1000 cycles
Threshold	600 – 900

Noise shifts latency; the separation survives in every tested environment.

An unprivileged clock cleanly tells a hit from a miss.

Paper § IV-B, Fig. 4 · pp. 5–6 · original distributions

Weapon #1 — Prime → Idle → Probe

1

Prime

Submit noop → cache my own completion-record page

→

2

Idle

Wait > one DSA op; a victim on this engine may evict it

→

3

Probe

Submit noop again, time it with rdtsc

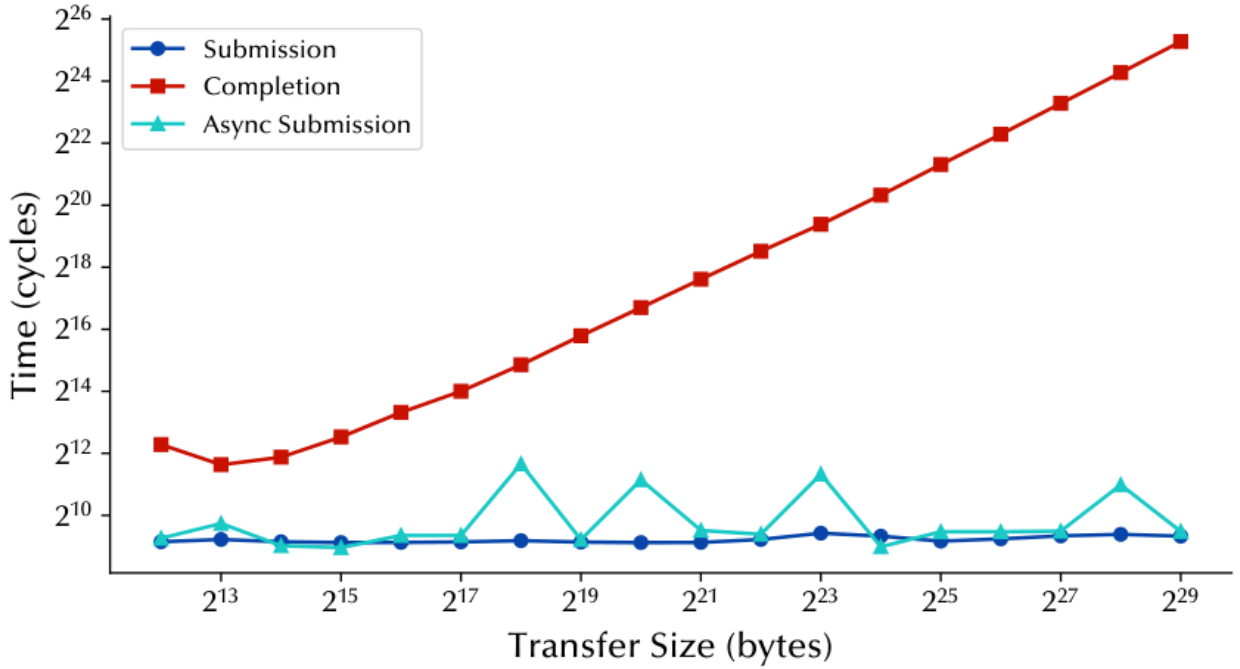
Fast probe → retained state. Slow probe → the victim moved.

Why it’s powerful — and what it cannot do

- **No address guessing**
  the attacker primes its own completion-record page
- **No eviction-set search**
  one slot — a single different page evicts it
- **A broad activity signal**
  meaning comes later, from timing patterns and a classifier

The primitive yields a heartbeat; the application supplies its meaning.

The dead end that wasn’t — a flag talks



Two observables

Submission flat ≈ 700 cycles

Completion grows with size

Queue-full a visible flag

The SWQ channel uses ENQCMD / EFLAGS.ZF — no timing threshold at all.

Paper § IV-C, Fig. 6 · pp. 6–7 · original latency plot

Learning to hold the queue hostage

Admission path

Portal → per-queue metadata check

Fixed-time occupancy test

Success or retry stays visible

Execution path

Work queue + batch buffer → arbiter

Arbiter prefers work over batch

So: use a long ordinary copy

The attacker holds the engine with a long ordinary copy, not batch priority.

The trap — leave exactly one seat open

Target state: effective occupancy N − 1



If the victim submits during the hold window, it consumes the last seat — the queue goes full.

The anchor must be large: contention only appeared for transfers ≥ 2^{25} bytes (32 MiB). Queue size is readable via accel-config without root.

Probe before the long copy completes and the queue starts draining.

Weapon #2 — Congest → Idle → Probe

1

Congest

Long memcpy + fillers → near-full SWQ

→

2

Idle

Hold the window; the victim may enqueue

→

3

Probe

Submit once, read EFLAGS.ZF

ZF = 0 → accepted (victim idle)

ZF = 1 → rejected (victim moved)

“Timer-free” describes the signal; the window still needs control.

Two weapons, two personalities

PROPERTY	DEVTLB	SWQ
Shared resource	same engine (queues may differ)	same shared work queue
Observed signal	completion latency	submission success / retry
Preparation	prime comp translation	hold queue near-full
Timer	rdtsc required	none
Footprint	light noop probes	heavy active congestion
Best at	covert bandwidth	keystroke timing

Both infer activity through the attacker’s own requests.

Act 4 of 6

04

What it steals

A covert channel proves the channel carries bits

Websites, SSH keystrokes, and the LLM next door

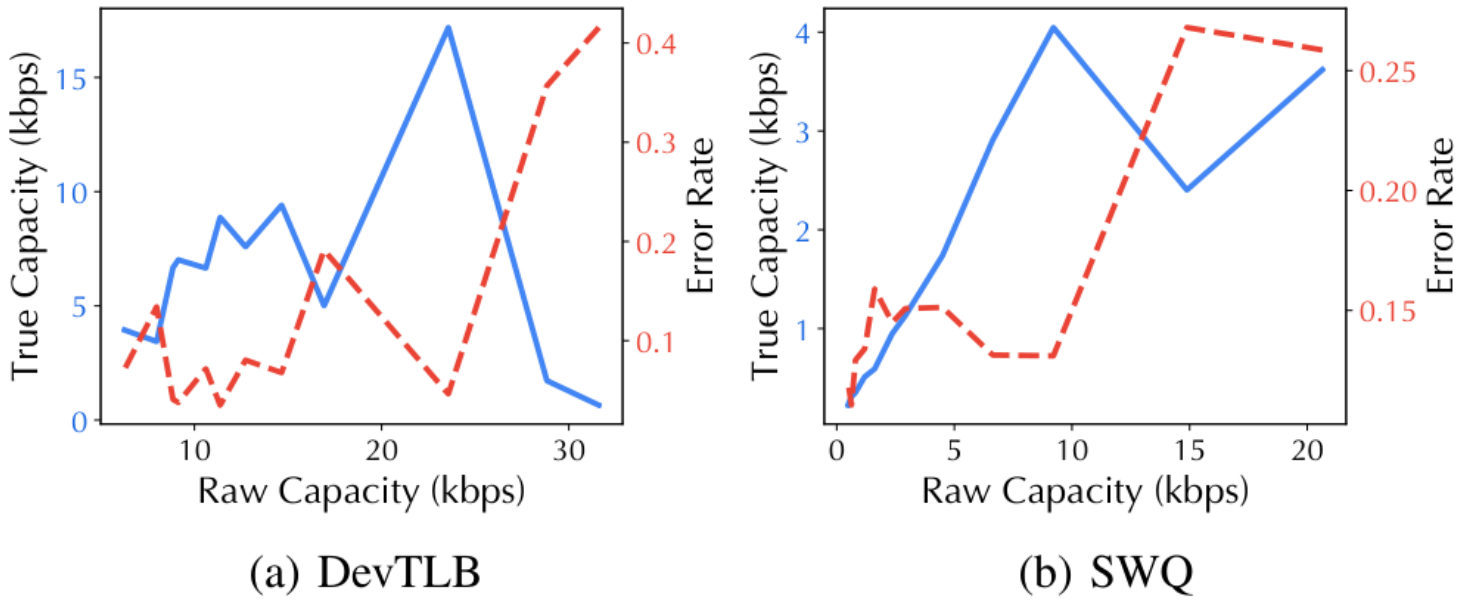
Strong numbers — quoted honestly, caveats and all

Can it even carry a message?

PHASE	RECEIVER	SENDER
Synchronize	detect preamble	transmit consecutive 1 bits
Prepare	prime DevTLB / congest SWQ	wait for data slot
Encode	open observation window	1 → submit · 0 → idle
Decode	time completion / read ZF	advance to next slot

Covert-channel feasibility is a separate claim from inference accuracy.

17.19 kbps — and a bandwidth / error tradeoff



17.19 kbps

DevTLB

4.63% bit error

4.02 kbps

SWQ

13.11% bit error

Faster raw signalling loses useful throughput as the error rate climbs.

What it steals

Attack #1 — websites leak through the stack



A fingerprint is learned from traffic-induced accelerator activity.

What it steals

The pipeline, laid bare — and it is closed-world

Sampling

probe every 10 µs

400 probes / slot

≈ 4 ms per slot

Model

2 × BiLSTM

attention + dropout

softmax classes

Dataset

100 sites × 200 traces

80 / 20 split

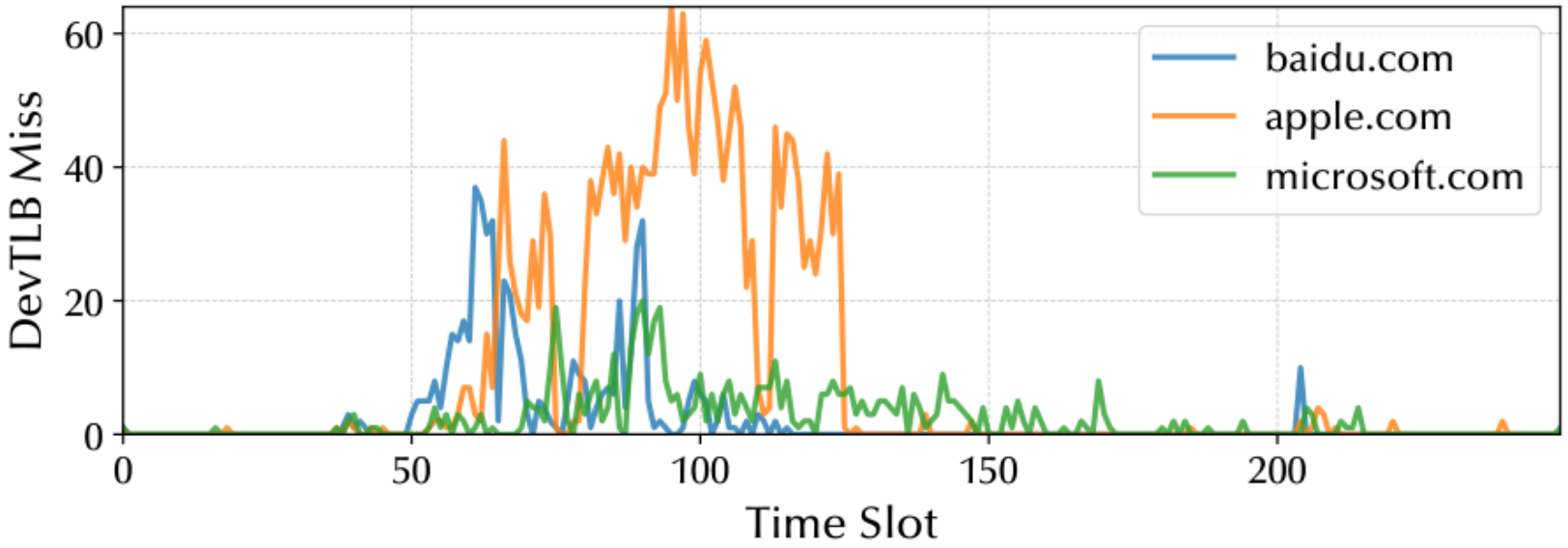
≈ 16,000 / 4,000

One feature per slot = the number of classified DevTLB misses.

The evaluated problem is closed-world classification over known labels.

Paper § VI-B · pp. 9–10 · dataset arithmetic from stated split

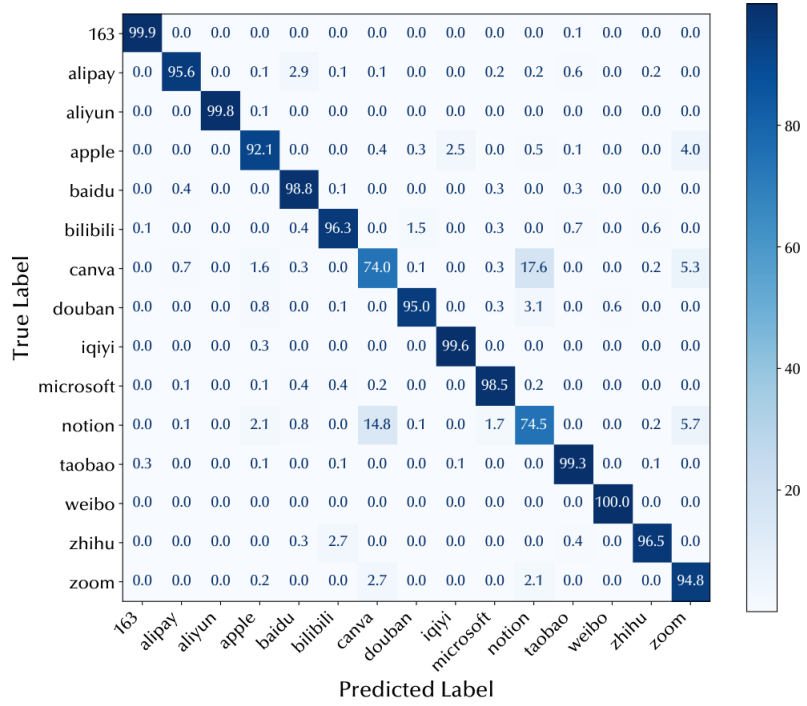
The traces really are different



Temporal shape is the feature; miss count is a proxy for DSA activity.

What it steals

85.73% — the number you must not misquote



85.73%

Full dataset

100 websites · 1% baseline

96.5%

Displayed subset

15 selected websites only

The headline is 85.73% on 100 sites; 96.5% is the 15-site subset.

Paper § VI-B, Fig. 11 · pp. 9–10 · original 15-site matrix

What it steals

Attack #2 — keystrokes, via transparent offload

User input

interactive SSH event

→

Packet handling

OpenSSH mem* calls

→

DTO offload

eligible calls → DSA

→

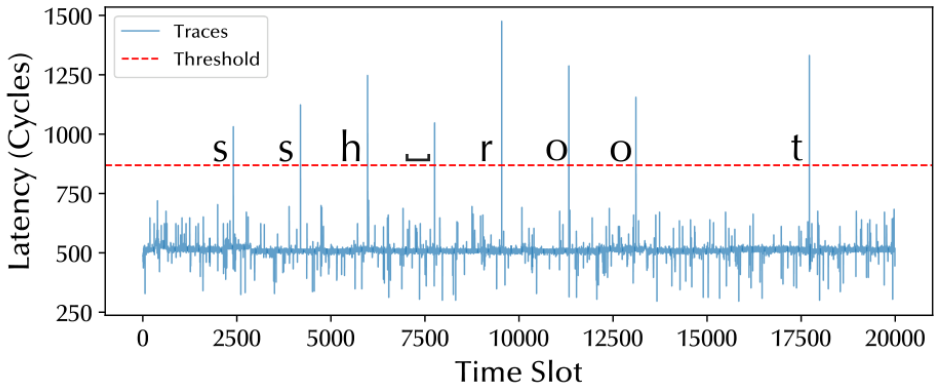
DSA observation

eviction or congestion

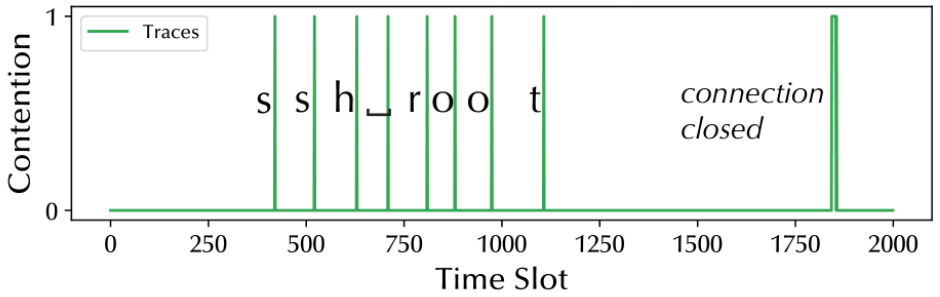
The demonstrated output is keystroke timing — not recovered command text.

What it steals

You can see the typing



(a) DevTLB traces during SSH input events.



(b) SWQ traces during SSH input events.

Read the panels

Top: completion latency

Bottom: queue contention

Labels: ground-truth input

Filtering affects event metrics; thresholds matter.

Both primitives reveal candidate keystroke events.

SWQ wins timing — and a caveat I owe you

PRIMITIVE	REPORTED F1	TIMESTAMP DEVIATION (Σ)
DevTLB	92.0%	5.29 ms
SWQ	**98.4%**	**1.21 ms**

Reporting caveat: the detection / FP / FN counts in § VI-C do not fully reconcile with the printed F1 (see backup).

Use the headline values with attribution; disclose the arithmetic gap.

What it steals

Attack #3 — fingerprinting the LLM next door

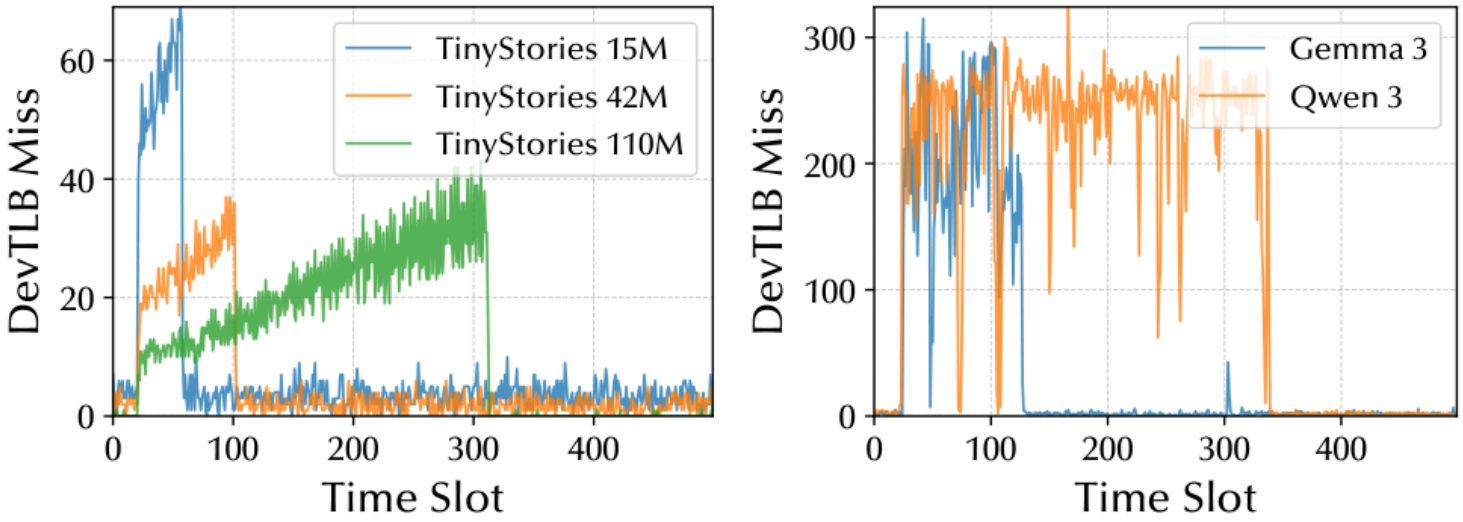
FAMILY	PARAMETER COUNTS	CHOICES
TinyStories	15M / 42M / 110M	3
LLaMA 2	7B	1
Gemma 3	1B / 4B	2
Qwen3	1.7B / 4B	2

50 traces per model · identical prompts · 80 / 20 split · 8 ms slots

The label is a model configuration — no weights or prompts are recovered.

What it steals

98.6% — impressive, and bounded



(a) TinyStories 15M/42M/110M (b) Gemma 3 and Qwen 3 (4B)

98.6%

Validation accuracy

8 model choices

Generality beyond identical prompts and fixed runtimes remains open.

Distinguishable configurations within a controlled dataset.

Paper § VI-D, Fig. 13 · p. 11 · original model traces

Act 5 of 6

05

Does it hold up?

Robust across quiet / noisy and local / cloud

Better than prior work on capacity and timing — not every axis

The clean fix costs the thing DSA is for

Does it hold up?

It barely flinches under stress

METRIC	LOCAL	NOISY LOCAL	CLOUD	NOISY CLOUD
DevTLB covert (kbps)	17.19	16.81	16.97	16.52
SWQ covert (kbps)	4.02	4.08	3.96	3.80
Website accuracy	85.7%	86.0%	85.5%	85.1%
DevTLB keystroke F1	92.0%	90.5%	93.4%	93.0%
SWQ keystroke F1	98.4%	98.2%	99.1%	98.9%
LLM accuracy	98.6%	98.6%	97.7%	98.0%

Robustness covers these stressors; broader tenant interference remains open.

Paper § VI-E, Table III · p. 11 · all six reported rows

Does it hold up?

Better on two axes — honestly, not all

WORK / MECHANISM	CAPACITY	BIT ERROR	KEYSTROKE F1	KEYSTROKE Σ	WEBSITE ACC.
IPI	3.45 kbps	18.9%	97.9%	6.15 ms	— (F1 80.4%)
DEVIOUS / IOTLB	2.16 kbps	2.18%	—	10.08 ms	98.9% [1]
This work / DevTLB	**17.19 kbps**	**4.63%**	**92.0%**	**5.29 ms**	**85.7%**
This work / SWQ	**4.02 kbps**	**13.11%**	**98.4%**	**1.21 ms**	**—**

[1] *DEVIOUS traces include GPU fingerprints; its IOTLB channel is since mitigated by PASID. IPI's keystroke F1 beats DevTLB; DEVIOUS's bit error beats both.*

Best on capacity (DevTLB) and keystroke timing (SWQ) — not on every axis.

Does it hold up?

The clean hardware fix costs the thing DSA is for

DevTLB

Partition replacement state by PASID

Cost: capacity · area · utilization

Goal: block cross-tenant eviction

SWQ

Make queue-state feedback privileged

Challenge: legit submission retries

Goal: block occupancy inference

A defense must constrain interference and feedback, not just permissions.

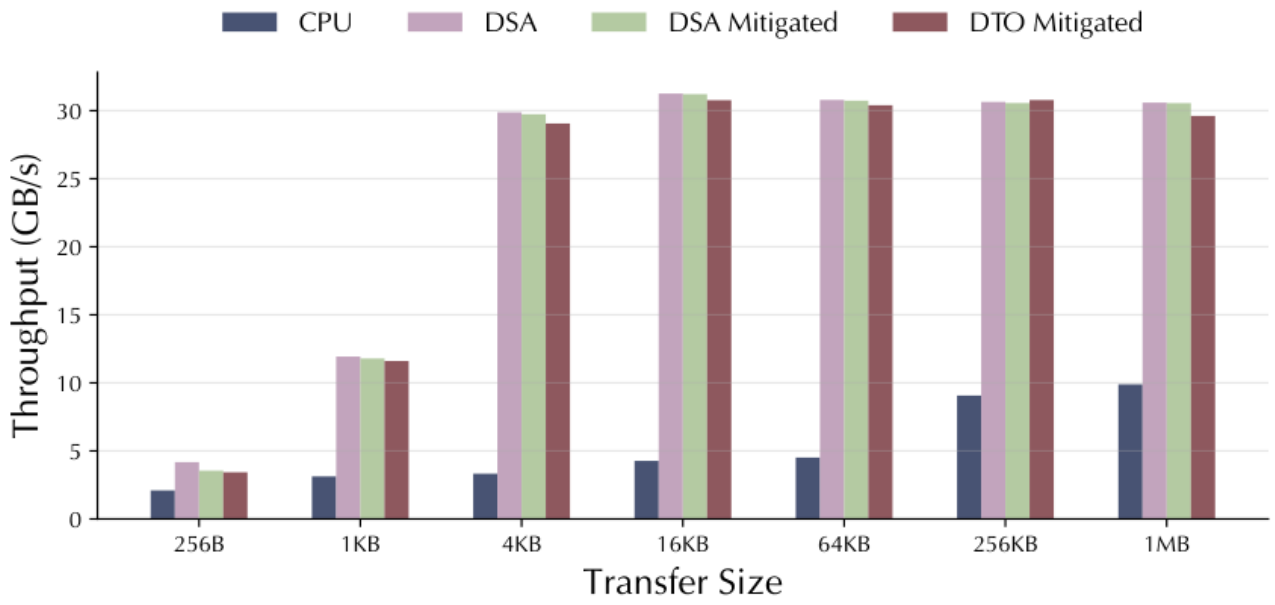
What an operator can do right now

EXPOSURE	BOUNDARY	SOFTWARE RESPONSE
DevTLB eviction	DSA engine	Allocate engines by trust domain
SWQ admission feedback	shared work queue	Avoid cross-tenant SWQ sharing
Sensitive-op offload	application path	Disable DSA on critical paths if needed

Separate queues do not close the shared-engine DevTLB channel.

Does it hold up?

They measured the cost — not the leftover leak



Maximum reported loss

Native DSA	15.7%
DTO	17.9%
At	256-byte transfers

The plot measures throughput overhead; residual leakage is not quantified.

Paper § VII, Fig. 14 · p. 12 · original throughput chart

Act 6 of 6

06

My verdict

The lasting contribution is the causal hardware analysis

Scope and reporting gaps qualify the headline claims

An isolation contract for shared accelerator state

My verdict

What’s genuinely excellent

- **Controlled reconstruction**
  page, field, queue, engine — each test is falsifiable
- **A real negative control**
  E2: separate engines eliminate the cross-VM DevTLB eviction
- **Two complementary interfaces**
  completion latency + admission feedback → distinct channels

The work turns an implementation detail into an isolation question.

My verdict

What I’d push back on

LIMIT	WHAT IT CHANGES
Provisioning and co-location	who can reach the target — SWQ discovery is sketched; engine binding is assumed
Closed-world / fixed-prompt datasets	how far classification accuracy generalizes
Active congestion and probing	victim impact and possible detectability
Metric inconsistencies / missing detail	confidence in exact quantitative claims

The shared-state leakage is well-supported; deployment reach needs more evidence.

My verdict

The experiments I'd want to see

Isolation study

Residual leakage + throughput

PASID-aware replacement

Generality study

Cross-day / cross-host

Unseen sites / prompts

Matched runtimes

Deployment study

Many concurrent tenants

Victim latency / retries

Probe detectability

Measure security and performance together, under the same configuration.

Presenter-proposed · not reported in the paper

Shared accelerator state belongs in the isolation model

01 Engine sharing exposes DevTLB replacement.

02 Queue sharing exposes admission feedback.

03 Application offload gives those signals meaning.

Discussion: what should an accelerator isolation contract guarantee?

Glossary

TERM	MEANING IN THIS PAPER
DSA	Intel Data Streaming Accelerator
ATS / TA	Address Translation Services / Translation Agent
ATC / DevTLB	device-side cache of address translations
PASID	process address-space ID; translation context
WQ / SWQ / DWQ	work queue / shared / dedicated work queue
DMWr / ENQCMD	deferrable memory write / descriptor submission
DTO	DSA Transparent Offload library

Descriptor layout — field meaning depends on opcode

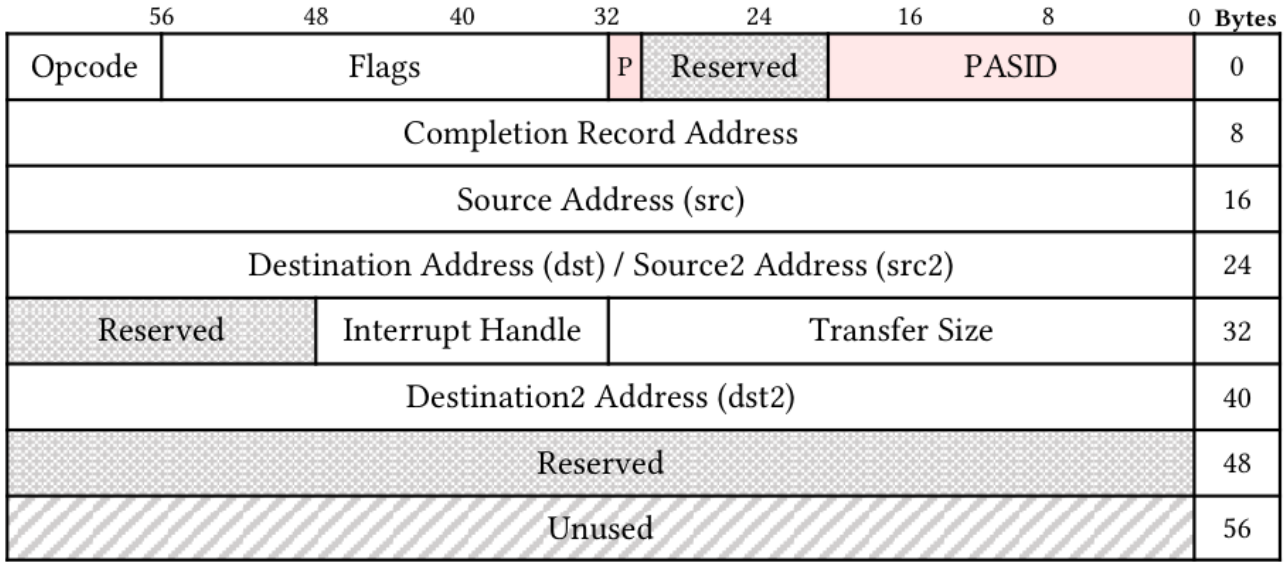


Fig. 3. DSA Descriptor format. Note that the descriptor is tagged with PASID to prevent other processes from executing it directly.

OFFSET	FIELD	SLOT
0	opcode / flags / PASID	header
8	completion record	comp
16	source	src
24	dest OR source2	dst / src2
40	second dest	dst2

Observation windows have different constraints

ASPECT	DEVTLB	SWQ
Prepare	own comp-page cached	effective occupancy ≈ N−1
Idle window	permit victim execution	shorter than holding copy
Probe	time own completion	read acceptance / retry
Calibration	hit / miss distributions	queue depth + drain
Confounder	other users of the slot	other submissions / completions

Keystroke count arithmetic needs clarification

FOR 512 GROUND-TRUTH EVENTS	DEVTLB	SWQ
Reported detections / FP	515 / 15	511 / 7
Implied TP = det − FP	500	504
Implied FN = 512 − TP	12	8
Printed FN / TP	**61 / 500**	**9 / 507**
Diagnostic F1 (fixed totals)	97.37%	98.53%

These calculations expose the inconsistency; they do not repair the experiment. Also: the abstract/contributions say 4.64% covert bit error; §VI-A and Table IV say 4.63%.

Sampling and dataset sizes

SETTING	WEBSITE TASK	LLM TASK
Sample interval	10 µs	10 µs
Samples per slot	400	800
Slot duration	4 ms	8 ms
Classes × traces	100 × 200	8 × 50
Total traces	20,000	400
80 / 20 split	16,000 / 4,000	320 / 80

Different metrics answer different questions

METRIC	QUESTION IT ANSWERS
Raw rate / true capacity	how fast are attempts / useful bits reported?
Bit error rate	how often does a decoded bit differ?
Top-1 accuracy	how often is the class correct?
Precision / recall	how many detections correct / events found?
F1 = 2TP / (2TP+FP+FN)	balance of precision and recall
Timestamp σ	how dispersed are timing differences?

The original cross-paper comparison table

TABLE IV
COMPARISON TO RECENT CROSS-CORE AND CROSS-VM ATTACKS.

		Website Fingerprinting		Keystroke Detection		Covert Channel		Mitigation
Work	**Co-location**	Accuracy	F_1 Score	F_1 Score	Std. Deviation	True Capacity	Bit Error Rate	PASID
IPI [51]	CPU	N/A	80.4%	97.9%	6.15 ms	3.45 kbps	18.9%	N/A
DEVIOUS [36]	Device	98.9%[1]	N/A	N/A	10.08 ms	2.16 kbps	2.18%	✓
(M)WAIT [65]	CPU	78%	78%			697 bps	0%	N/A
This work (DevTLB)	Device	85.7%	N/A	92.0%	5.29 ms	17.19 kbps	4.63%	✗
This work (SWQ)	Device			98.4%	1.21 ms	4.02 kbps	13.11%	✗

[1] The trace collected in DEVIOUS includes GPU fingerprints.

Reproduction requires a configured DSA platform

- **Hardware & provisioning**

  DSA-capable Xeon; VT-d / IDXD; admin configuration + device access

- **Victim-specific software**

  VPP / DPDK for websites; DTO-enabled SSH and inference workloads

- **Artifact organization**

  playground → characterization; attack → traces; evaluate → metrics

github.com/hkustgz-secarch/dsassassin · DOI 10.5281/zenodo.17799273

Likely questions and concise answers

QUESTION	ANSWER
Does encryption stop it?	content protection does not remove activity patterns
Do separate CPU cores help?	the observations arise in shared DSA resources
Do separate queues help?	SWQ: yes; DevTLB: also separate the engines
Are typed characters recovered?	it evaluates event timing, not full text recovery
Would timer removal solve it?	SWQ reads admission feedback without a duration
Is random insertion a proven fix?	overhead is measured; residual leakage is not

Sources and provenance

- **Primary paper**

  Chen, Li, Deng, Wang, Chen. DSASSASSIN. HPCA 2026

- **Author-hosted PDF**

  i.benx.dev/assets/hpca26dsassassin.pdf

- **Artifact**

  github.com/hkustgz-secarch/dsassassin

- **Archive + record**

  doi.org/10.5281/zenodo.17799273 · 2026.hpca-conf.org

Primary source: user-supplied hpca26dsassassin.pdf